

CPU Scheduling Algorithms

Analysis & Conclusion — Round Robin vs SRTF

This report covers the **required analysis questions** and **final conclusion** for the CPU Scheduling project. All numeric values are derived directly from the simulator code (Round Robin and SRTF implementations) run against the five defined scenarios.

Section 1 — Required Analysis Questions

Q1 How does quantum size affect Round Robin performance?

Scenario B uses the same three processes (P1 burst=20, P2 burst=3, P3 burst=4) with two different quantum values, isolating the effect of quantum size alone.

$q = 2 \rightarrow \text{Avg WT} = 6.67 \mid \text{Avg TAT} = 15.67 \mid \text{Avg RT} = 2.0$

$q = 8 \rightarrow \text{Avg WT} = 8.67 \mid \text{Avg TAT} = 17.67 \mid \text{Avg RT} = 6.33$

With **q=2**, the CPU is shared in smaller slices so short processes (P2, P3) complete much earlier. Response time for P2 drops from 8 (q=8) to 2 (q=2) — a 4× improvement. Average WT improves by 23% and TAT by 11%.

With **q=8**, P1 monopolises the CPU for 8 units on each visit. P2 and P3 must wait an entire P1 slice before their first execution, pushing RT to 8 and 11 respectively.

Conclusion: a smaller quantum improves interactivity and fairness for short processes, but increases context-switch overhead. A larger quantum approaches FCFS behaviour and is efficient for CPU-bound workloads but poor for responsiveness.

Q2 Why does SRTF consistently outperform RR in average WT and TAT?

SRTF is a **preemptive optimal** algorithm: at every time unit it selects the process with the shortest remaining burst. This provably minimises average waiting time — no other scheduling algorithm can achieve a lower average WT.

Evidence across all five scenarios:

Scenario A RR Avg WT = 13.50 $\rightarrow$ SRTF Avg WT = 6.50 (2.1× better)

Scenario B q=2 RR Avg WT = 6.67 $\rightarrow$ SRTF Avg WT = 3.33 (2.0× better)

Scenario B q=8 RR Avg WT = 8.67 $\rightarrow$ SRTF Avg WT = 3.33 (2.6× better)

Scenario C RR Avg WT = 3.80 $\rightarrow$ SRTF Avg WT = 1.60 (2.4× better)

Scenario D RR Avg WT = 6.50 $\rightarrow$ SRTF Avg WT = 3.00 (2.2× better)

RR wastes time cycling through *all* processes in fixed slices regardless of how much work remains. SRTF avoids this by always prioritising the process closest to completion, reducing the total time other processes spend waiting.

Q3 When does Round Robin beat SRTF in Response Time (RT)?

Despite SRTF's superiority in WT and TAT, **RR achieves better average RT** in several scenarios:

Scenario A RR Avg RT = 3.00 SRTF Avg RT = 4.25 → RR wins

Scenario B $q=2$ RR Avg RT = 2.00 SRTF Avg RT = 3.33 → RR wins

This occurs because RR **guarantees bounded first access**: every process receives CPU time within at most $(n \times \text{quantum})$ units of its arrival. No process waits more than one full cycle for its first slice.

SRTF can delay a long-burst process (e.g. P3 burst=9 in Scenario A) until *all* shorter processes finish. In Scenario A, P3's first CPU access under SRTF is at $t=17$ (RT=15), while RR grants it CPU at $t=6$ (RT=4).

Trade-off summary: RR = bounded, fair first response. SRTF = optimal total wait.

Q4 Starvation: can SRTF starve long processes?

Yes — SRTF can cause **indefinite starvation** for long-burst processes in a system with continuous short-burst arrivals.

Scenario D illustrates this: P1 (burst=20) starts at $t=0$ and runs for 2 units. Then P2 (burst=3, arrival=2), P3 (burst=3, arrival=4), and P4 (burst=3, arrival=6) each preempt P1 immediately because their remaining time is shorter. P1 does not resume until $t=11$ and finishes at $t=29$.

In a real system where short processes arrive continuously, P1 would never regain the CPU — a textbook starvation scenario.

Round Robin prevents starvation entirely: every process is guaranteed CPU access within $(n \times \text{quantum})$ time units, regardless of burst length. This is one of RR's key advantages in general-purpose operating systems.

Q5 Scenario C: why do short processes benefit far more from SRTF?

Scenario C pairs one long process (P1, burst=10) with four very short ones (P2–P5, bursts 1–2). This maximises SRTF's advantage.

Under **RR ($q=3$)**: P1 executes first for 3 units before any short process runs, delaying all of them unnecessarily.

Under **SRTF**: P1 is preempted the moment P2 arrives at $t=1$ (P2 remaining=1 < P1 remaining=9). Every short process runs to completion before P1 resumes.

Avg WT: RR = 3.80 SRTF = 1.60 → 2.4× improvement

Avg RT: RR = 2.60 SRTF = 0.40 → 6.5× improvement

The **6.5× improvement in response time** shows how powerfully SRTF rewards short-burst workloads when they are mixed with a long-running process.

Q6 Validation (E-series): what edge cases does the validator catch?

The E-series scenarios deliberately test input boundaries in **validator.py**:

- **E(p2) — arrival = -1:** Rejected. The validator correctly rejects negative arrival times with: *"Arrival time of P1 cannot be negative"*.
- **E(p3) — burst = -3 or 0:** Rejected. Burst time must be strictly positive (*"Burst time must be positive"*).
- **E(p4) — duplicate process IDs, P1 and P1** are rejected, IDs must be different not duplicated.
- **E(p5) — quantum = 0: Potential bug.** The validator does not check for $\text{quantum} \leq 0$. Quantum validation only happens in **main_ui.py** inside **run()**, which catches non-integer input but accepts $\text{quantum}=0$. Passing $\text{quantum}=0$ to **round_robin()** causes an **infinite loop** because $\text{min}(0, \text{remaining}) == 0$, so **remaining** never decrements.

Section 2 — Required Conclusion

Across all five scenarios, **SRTF consistently outperforms Round Robin in average Waiting Time and Turnaround Time** by a factor of approximately 2×. In contrast, **Round Robin frequently achieves better average Response Time** due to its guaranteed time-sharing nature. The choice between the two algorithms represents a fundamental scheduling trade-off.

Round Robin

- ✓ Best average Response Time
- ✓ Guaranteed fairness — no starvation
- ✓ Suitable for interactive / time-sharing systems
- ✗ Higher average Waiting Time and TAT
- ✗ Performance depends on quantum selection

Ideal for multi-user OS where fast first response matters more than throughput.

SRTF

- ✓ Optimal average Waiting Time (provably minimal)
- ✓ Best average Turnaround Time in all scenarios
- ✓ Ideal for batch / throughput-oriented workloads
- ✗ Risk of starvation for long-burst processes
- ✗ Requires burst time known in advance

Ideal for systems with known burst times targeting maximum throughput.

Summary Statistics Across All Scenarios

Scenario	RR Avg WT	SRTF Avg WT	RR Avg TAT	SRTF Avg TAT	RR Avg RT	SRTF Avg RT
Scenario A	13.5	6.5	20.0	13.0	3.0	4.25
Scenario B (q=2)	6.67	3.33	15.67	12.33	2.0	3.33
Scenario B (q=8)	8.67	3.33	17.67	12.33	6.33	3.33
Scenario C	3.8	1.6	7.0	4.8	2.6	0.4
Scenario D	6.5	3.0	13.75	10.25	1.25	0.75

Key Takeaway

Neither algorithm dominates across all metrics. SRTF minimises total waiting and turnaround time but risks starvation and requires knowledge of burst times in advance — making it more theoretical than practical. Round Robin trades some efficiency for guaranteed fairness and bounded response time, which is why it (and its variants) form the basis of scheduling in most real-world general-purpose operating systems. In practice, hybrid approaches that combine priority levels with round-robin within each level (as seen in Linux's CFS and Windows thread scheduling) are used to capture the benefits of both.